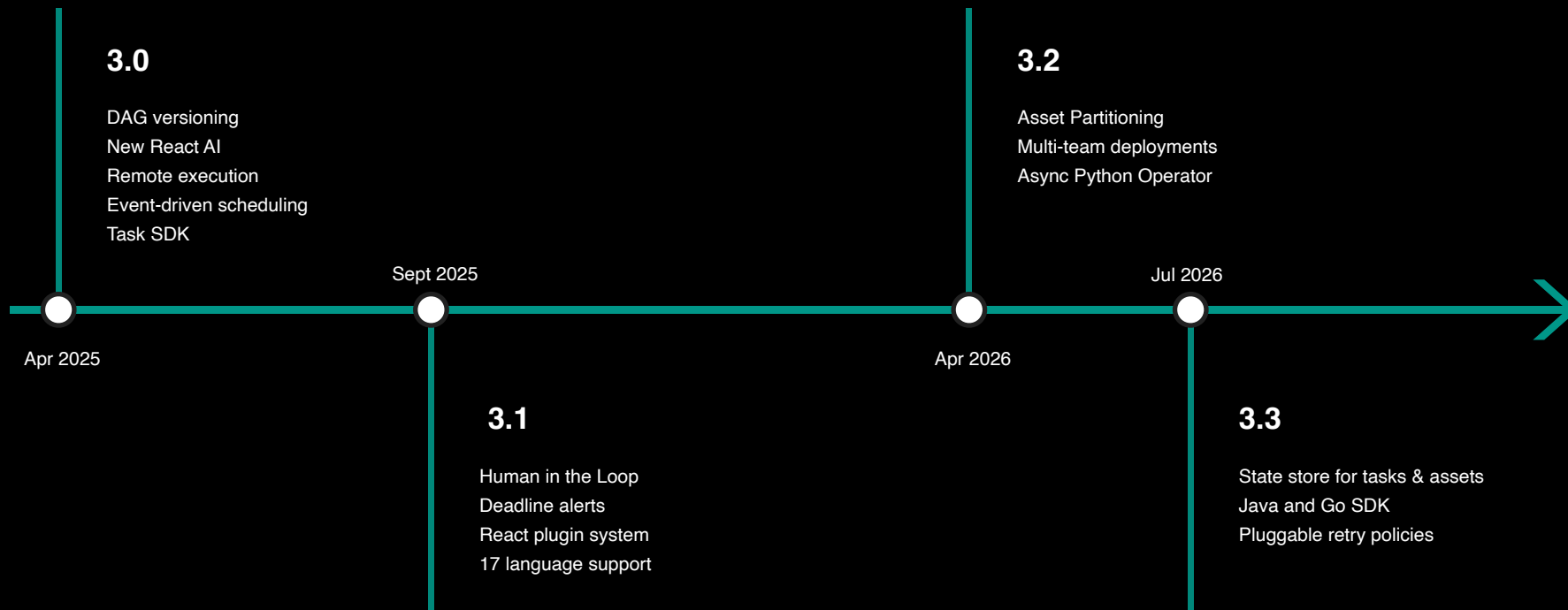


Streamlining Your Airflow Upgrade: 2.x -> 3



Ankit Chaurasia, Data Platform Engineer at Apple

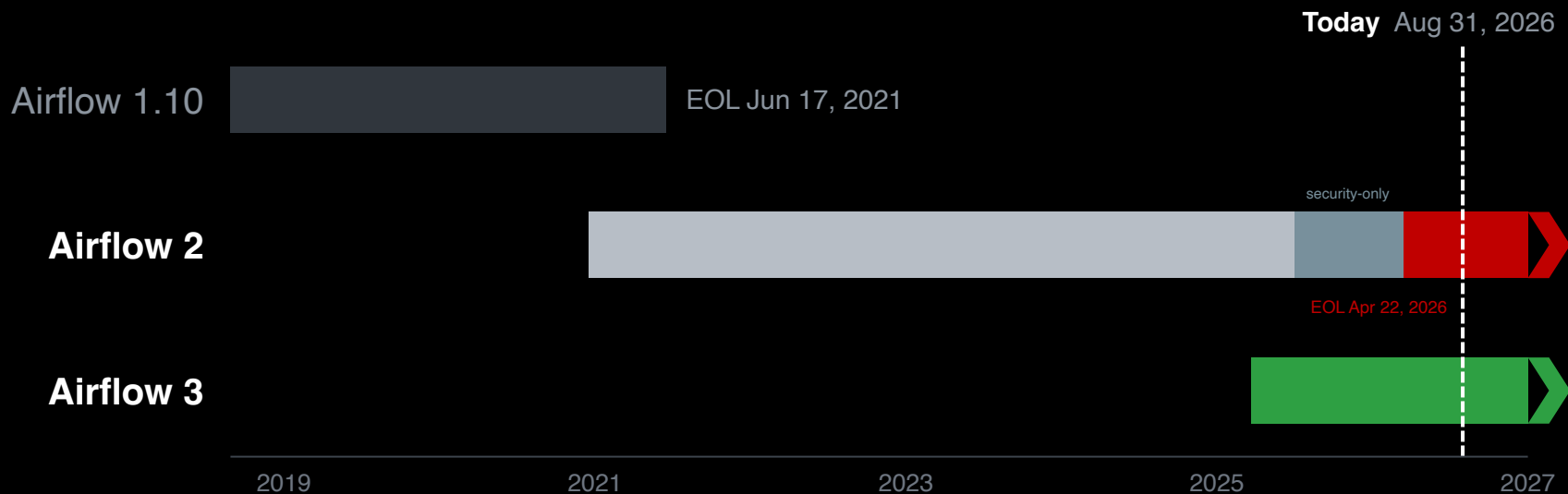
Last year: 3.0 Today: 3.3.0



Why upgrade?



Airflow 3 is only maintained version receiving CVE patches.



Meet our Airflow 2 DAG



```
import pendulum
from airflow import DAG
from airflow.models import DagRun
from airflow.operators.python import PythonOperator
from airflow.utils.session import provide_session

@provide_session
def load(execution_date, dag_run, session=None, **ctx):
    rows = (
        session.query(DagRun)
        .filter(DagRun.dag_id == dag_run.dag_id)
        .all()
    )
    day = execution_date.strftime("%Y-%m-%d")
    write_partition(rows, day)

with DAG(
    dag_id="daily_revenue",
    start_date=pendulum.datetime(2026, 7, 1, tz="UTC"),
    schedule_interval="@daily",
    catchup=False,
) as dag:
    PythonOperator(task_id="load", python_callable=load)
```

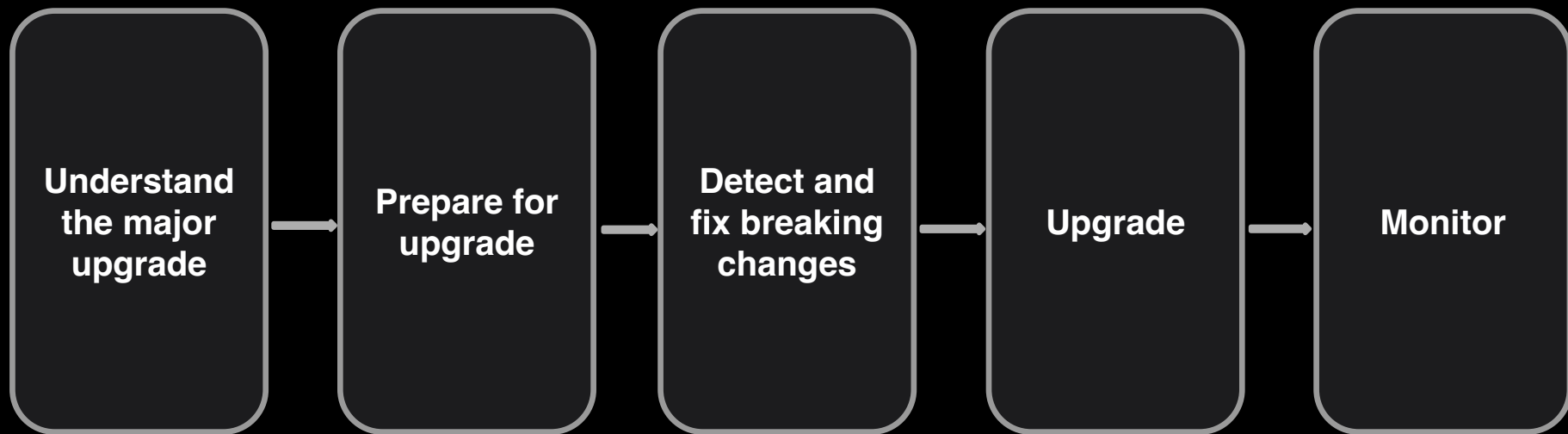
Old import paths

Direct DB access

Execution date

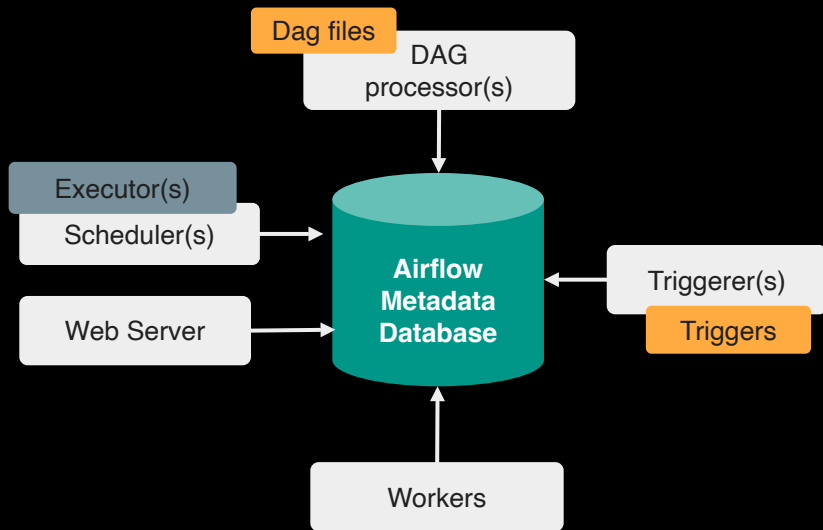
Schedule interval

Your upgrade journey

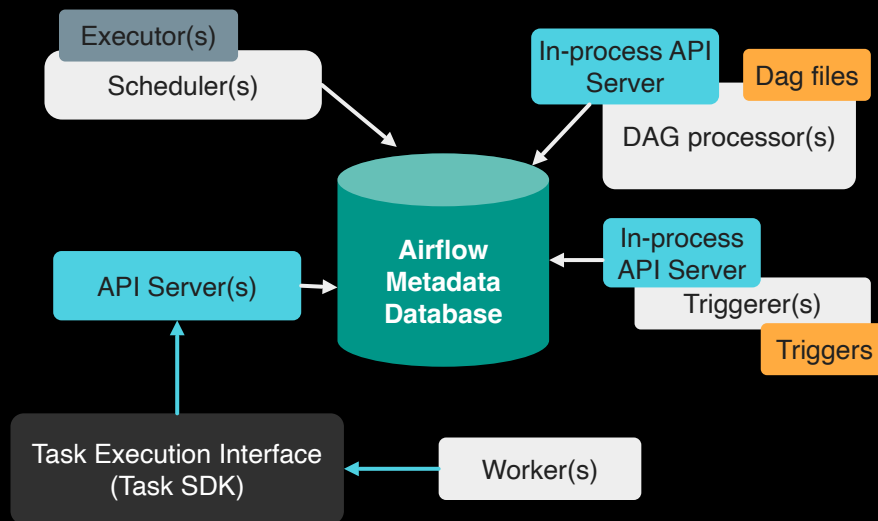


First, understand the change

Architecture change



Airflow 2



Airflow 3

Removed and replaced features



- SubDags
- SLAs
- SequentialExecutor and DebugExecutor
- CeleryKubernetesExecutor and LocalKubernetesExecutor
- DummyOperator
- SimpleHTTPOperator

Prepare for upgrade

Prerequisites



**Airflow version
2.11.x**

Python ≥ 3.10

**PostgreSQL 13+
or MySQL 8+**

SQLAlchemy 2

**Update all
providers**

**Install standard
providers**

Clean and back up your existing Airflow instance



**Back up the
metadata
database**

**Clean the
database**

**Resolve DAG
errors**

Detect and fix breaking changes

Ruff AIR rules



```
$ pip install "ruff>=0.15.17"
$ ruff check dags/ --select AIR3 <path_to_your_dag_code>
```

AIR30x = breaking

AIR31x = recommended

AIR301

AIR311

AIR302

AIR312

AIR303

AIR304

Detect breaking changes



```
from airflow.decorators import dag
from airflow.operators.bash import BashOperator

@dag(
    start_date=None,
    schedule=None,
    catchup=False,
    fail_stop=True,
)
def test_ruff_linter_dag():
    BashOperator(
        task_id="my_task",
        bash_command="echo 'Hello, World!'",
    )

test_ruff_linter_dag()
```

```
> ruff check dags/ --select AIR3
AIR311 'airflow.decorators.dag' is removed in Airflow 3.0; It still works in Airflow 3.0 but is expected to be removed in a future version.
→ dags/exampledag.py:5:2
5 | @dag(
  | ^^^
6 |     start_date=None,
7 |     schedule=None,
  |
help: 'dag' has been moved to 'airflow.sdk' since Airflow 3.0 (with apache-airflow-task-sdk≥1.0.0).

AIR301 [*] 'fail_stop' is removed in Airflow 3.0
→ dags/exampledag.py:9:5
7 |     schedule=None,
8 |     catchup=False,
9 |     fail_stop=True,
  |     ^^^^^^^^^^^
10 | )
11 | def test_ruff_linter_dag():
  |
help: Use 'fail_fast' instead
8 |     catchup=False,
  -     fail_stop=True,
9 +     fail_fast=True,
10 | )

AIR312 'airflow.operators.bash.BashOperator' is deprecated and moved into 'standard' provider in Airflow 3.0; It still works in Airflow 3.0
→ dags/exampledag.py:12:5
10 | )
11 | def test_ruff_linter_dag():
12 |     BashOperator(
  |     ^^^^^^^^^^^
13 |         task_id="my_task",
14 |         bash_command="echo 'Hello, World!'",
  |
help: Install 'apache-airflow-providers-standard'≥0.0.1 and use 'BashOperator' from 'airflow.providers.standard.operators.bash' instead.

Found 3 errors.
[*] 1 fixable with the '--fix' option (2 hidden fixes can be enabled with the '--unsafe-fixes' option).
```

Fix Ruff errors



```
> ruff check dags/ --select AIR300 --fix --unsafe-fixes
Found 3 errors (3 fixed, 0 remaining).
```

```
from airflow.sdk import dag
from airflow.providers.standard.operators.bash import BashOperator

@dag(
    start_date=None,
    schedule=None,
    catchup=False,
    fail_fast=True,
)
def test_ruff_linter_dag():
    BashOperator(
        task_id="my_task",
        bash_command="echo 'Hello, World!'",
    )

test_ruff_linter_dag()
```

Hard ones which Ruff can't fix

Remove direct database access



Airflow 2

Direct DB access - ORM



```
# Task queries the metadata DB directly
from airflow.decorators import dag, task
from airflow.models import DagModel
from airflow.settings import Session

@dag(schedule=None)
def list_dags_dag():
    @task
    def list_dags():
        with Session() as session:
            dags = (session.query(DagModel)
                    .limit(10).all())
            print(dags)
    list_dags()
list_dags_dag()
```

Airflow 3

Use airflow REST API - v2



```
from airflow.sdk import dag, task
import requests, os
HOST = "http://api-server:8080"
@dag(schedule=None)
def list_dags_dag():
    @task
    def list_dags():
        token = requests.post(f"{HOST}/auth/token",
                              json={"username": os.getenv("AF_USER"),
                                    "password": os.getenv("AF_PASS")},
                              ).json()["access_token"]
        dags = requests.get(f"{HOST}/api/v2/dags",
                             headers={"Authorization": f"Bearer {token}"},
                             params={"limit": 10}).json()
        print(dags)
    list_dags()
list_dags_dag()
```

XCom pickling removed – write a custom backend



Airflow 2

Pickle workaround



```
# Pickle anything via config
AIRFLOW__CORE__ENABLE_XCOM_PICKLING=True

@task
def push():
    return np.array([1, 2, 3])
```

Airflow 3

Custom XCom backend



```
# Setting removed – same task now fails:
# TypeError: ndarray not JSON serializable
from airflow.sdk.bases.xcom import BaseXCom

class MyXCom(BaseXCom):
    @staticmethod
    def serialize_value(value, **kwargs):
        if isinstance(value, np.ndarray):
            return json.dumps(
                {"v": value.tolist()}).encode()
        return BaseXCom.serialize_value(value)
```

Custom trigger – hook is now async



Airflow 2

Sync call in async run()



```
# Sync hook inside the trigger
class MySQLTrigger(BaseTrigger):
    async def run(self):
        while True:
            hook = MySQLHook(
                conn_id=self.conn_id)
```

Airflow 3

Wrapped with sync_to_async



```
# Wrap the hook (asgiref)
from asgiref.sync import sync_to_async

class MySQLTrigger(BaseTrigger):

    async def run(self):
        while True:
            hook = await sync_to_async(
                MySQLHook)(conn_id=self.conn_id)
```

SubDAGs removed



Airflow 2

SubDag



```
# SubDagOperator removed in Airflow 3
from airflow.operators.subdag import (
    SubDagOperator)

SubDagOperator(
    task_id="process",
    subdag=make_child_dag(...),
)
```

Airflow 3

TaskGroup / asset-chained Dags



```
# Same grouping with a TaskGroup
from airflow.sdk import TaskGroup

with TaskGroup("process") :
    extract() >> transform() >> load()

# or: small Dags on asset schedules
@dag(schedule=[Asset("raw_orders")])
```

Scheduling default changed



Airflow 2

Implicit defaults



```
@dag()          # unset meant:
# schedule = timedelta(days=1)
# catchup = True

schedule="0 0 * * *"
# CronDataIntervalTimetable:
# logical_date = interval start,
# data_interval spans the period
```

Airflow 3

New meanings + revert switches



```
@dag()          # unset now means:
# schedule = None    -> never runs!
# catchup = False

schedule="0 0 * * *"
# CronTriggerTimetable:
# data_interval_start == run_after

# Revert globally (airflow.cfg / env):
AIRFLOW__SCHEDULER__CATCHUP_BY_DEFAULT=True
AIRFLOW__SCHEDULER__CREATE_CRON_DATA_INTERVALS=True
```

Removed context keys



Airflow 2

Keys that disappear



```
{{ execution_date }}
```

```
{{ tomorrow_ds }} / {{ yesterday_ds }}
```

```
{{ prev_ds }} / {{ next_ds }}
```

```
context["conf"]
```

```
templates_dict
```

```
{{ triggering_dataset_events }}
```

Airflow 3

Closest replacement



```
dag_run.logical_date
```

```
compute: ds ± timedelta(days=1)
```

```
prev_start_date_success /
```

```
timetable.next_dagrun_info()
```

```
from airflow.sdk import conf # 3.2+
```

```
params
```

```
triggering_asset_events
```

Logical date vs execution date



Airflow 2

ds = data_interval_start



```
parameters={
    "YESTERDAY": "{{ ds }}",
    "TODAY": "{{ macros.ds_add(ds, 1) }}",
}
```

templated values - correct

```
{'YESTERDAY': '2025-11-16',
 'TODAY':     '2025-11-17'}
```

Airflow 3

ds = run_after — shifted +1 day!



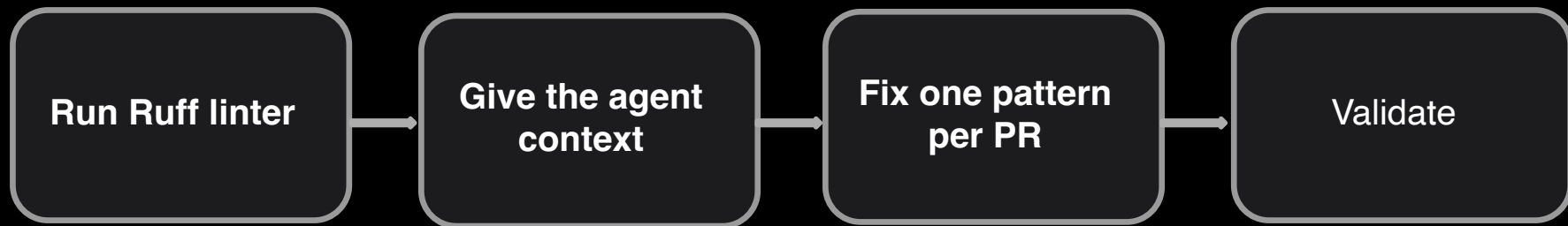
unchanged template - wrong partition

```
{'YESTERDAY': '2025-11-17',
 'TODAY':     '2025-11-18'}
```

Fix — anchor on "today", look back

```
parameters={
    "YESTERDAY": "{{ macros.ds_add(ds, -1) }}",
    "TODAY":     "{{ ds }}",
}
```

Use AI agents for the manual fixes



Airflow config changes



**Default values
changed**

**Renamed
options**

**Removed
options**

**Invalidated
previously valid
options**

airflow config update



The following are the changes in airflow config:

- [BREAKING] Updated default value of 'core/executor' from 'SequentialExecutor' to 'LocalExecutor'.
- [BREAKING] Removed 'logging/log_filename_template' from configuration.
- [BREAKING] Renamed 'webserver/web_server_host' to 'api/host'.
- [BREAKING] Renamed 'webserver/web_server_port' to 'api/port'.
- [BREAKING] Renamed 'webserver/workers' to 'api/workers'.
- [BREAKING] Renamed 'webserver/web_server_ssl_cert' to 'api/ssl_cert'.
- [BREAKING] Renamed 'webserver/web_server_ssl_key' to 'api/ssl_key'.
- [BREAKING] Renamed 'webserver/access_logfile' to 'api/access_logfile'.
- [BREAKING] Updated default value of 'scheduler/catchup_by_default' from 'True' to 'False'.
- [BREAKING] Renamed 'scheduler/dag_dir_list_interval' to 'dag_processor/refresh_interval'.
- [BREAKING] Renamed 'triggerer/default_capacity' to 'triggerer/capacity'.

Dry-run is mode enabled. To apply above airflow.cfg run the command with `--fix`.

airflow config update is available from Airflow >=2.11.1

Upgrade locally first, then in production



LOCAL

upgrade and test on your machine

```
# pip install, or update the docker-compose image
AIRFLOW_VERSION=3.x
```

```
airflow api-server          # replaces webserver
airflow dag-processor
airflow db migrate
```

```
# run the updated Dags here to test them
```

PRODUCTION

upgrade the deployment

```
# back up the metadata database first
# upgrade during a low-volume window
```

```
airflow db clean
AIRFLOW_VERSION=3.x          # same as local
```

```
airflow api-server          # replaces webserver
airflow dag-processor
airflow db migrate           # large DBs: up to 2 h
```

Update your integrations



Airflow 2

Basic auth, v1



```
# Any script, CI job, dashboard...
curl http://af:8080/api/v1/dags \
  -H "Accept: application/json" \
  --user "admin:admin"
```

Airflow 3

JWT token, v2



```
# Get a token, then call v2
TOKEN=$(curl -X POST \
  http://af:8080/auth/token \
  -d '{"username":"admin","password":"admin"}' \
  | jq -r .access_token)

curl http://af:8080/api/v2/dags \
  -H "Authorization: Bearer $TOKEN"
```

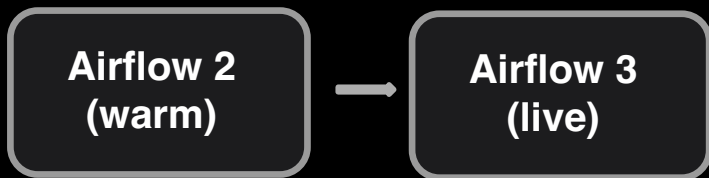
Live demo



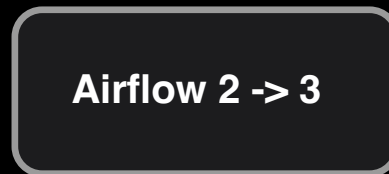
Production upgrade strategy



Blue-green (recommended)



In-place upgrade



Resources



- [Apache Airflow documentation](#)
- [Ruff - Airflow \(AIR\) rules](#)
- [Apache Airflow Slack channel #airflow-upgrade](#)



Questions?

